

Trainee Management System

Detailed Real-Time Project Specification. A genuine internal application — not a tutorial. Expand the scope gradually. Do not try to build the entire system at once.

TYPE Real-Time Full-Stack Project	BACKEND Python + FastAPI	FRONTEND React + TypeScript
DATABASE MySQL	PROGRAM DTPP Trainee Program	PROJECT CODE DTPP-TMS-001

01 Project Overview

The Trainee Management System (TMS) is a real-world internal application for managing trainees throughout the DTPP program. The system will replace manual trainee records and provide a centralized platform for:

Trainee registration	Batch management	Mentor assignment	Attendance	Tasks	Assignments	Submissions	Evaluation
Feedback	Progress tracking	Reports					

MAIN OBJECTIVE

Register → Assign Batch → Assign Mentor → Training → Attendance → Tasks → Assignments → Evaluation → Feedback → Completion

02 Project Goals

BUSINESS GOAL Provide Deshmukh Technologies with a simple system to manage DTPP trainees.	TECHNICAL GOAL Teach trainees how to develop a complete full-stack application.
PROFESSIONAL GOAL Teach the real software-development lifecycle.	LEARNING GOAL Requirement → Design → Development → Testing → Git → Code Review → QA → Deployment

03 User Roles

3.1 ADMIN Complete access. Admin can: - Login - Create users - Add / edit trainees - Deactivate trainees - Create batches - Assign mentors - Assign trainees - View attendance, tasks, evaluations, reports	3.2 MENTOR Manages assigned trainees. Mentor can: - Login - View assigned trainees - Mark attendance - Create and assign tasks - Review submissions - Evaluate trainees - Give feedback - View trainee progress	3.3 TRAINEE Trainees have limited access. Trainee can: - Login - View profile, batch, mentor - View attendance and tasks - Submit assignments - View feedback and evaluation - Track progress
---	---	---

04 Main Modules

```
TMS
├── Authentication
├── User Management
├── Trainee Management
├── Mentor Management
├── Batch Management
├── Attendance
├── Task Management
├── Assignment Management
├── Submission Management
├── Evaluation
├── Feedback
├── Dashboard
└── Reports
```

05 Module 1 — Authentication

Features: Login · Logout · Password hashing · JWT authentication · Role-based authorization · Protected APIs.

Username / Email → Password → Authentication → JWT Token → Dashboard

Roles: **ADMIN** **MENTOR** **TRAINEE**

06 Module 2 — Trainee Management

This is the first major module.

Trainee information

Trainee ID	Full Name
Email	Mobile
Date of Birth	Gender
Education	Address
Joining Date	Batch / Mentor
Status	ACTIVE · INACTIVE · COMPLETED · DROPPED

LIST, SEARCH, FILTER

View: Name, ID, Batch, Mentor, Status. Search by Name, Trainee ID, Email, Mobile. Filter by Batch, Mentor, Status.

DEACTIVATE

Do not immediately delete important records. Prefer **ACTIVE** → **INACTIVE**.

07 Module 3 — Batch Management

A batch represents a group of trainees. Examples: DTTP-2026-01, DTTP-2026-02, DTTP-2026-03.

Batch information: Batch ID, Batch Name, Start Date, End Date, Mentor, Capacity, Status.

Features: create batch, edit batch, view batch, assign mentor, add trainees, remove trainee, view batch members.

08 Module 4 — Mentor Management

Mentor information: Mentor ID, Name, Email, Mobile, Department, Specialization, Status.

Features: add mentor, edit mentor, view mentor, assign trainees, view assigned trainees, activate/deactivate mentor.

09 Module 5 — Attendance

Mentors record trainee attendance. Status: **PRESENT** **ABSENT** **LATE** **LEAVE**

Attendance record: Date, Trainee, Batch, Status, Marked By, Remarks.

Features: mark attendance, edit attendance, view daily / monthly / individual attendance, calculate attendance percentage.

```
Total Working Days = 25
Present = 23
Attendance = 92%
```

10 Module 6 — Task Management

Mentors create tasks for trainees. Task information: Task ID, Title, Description, Assigned To, Created By, Priority, Deadline, Status.

Priority: LOW MEDIUM HIGH

Status: TODO IN_PROGRESS SUBMITTED UNDER_REVIEW COMPLETED

```
Mentor → Create Task → Trainee → Start Task → Submit
→ Mentor Review → Feedback → Completed
```

11 Module 7 — Assignment Management

Assignments are larger learning activities. Example: Build a FastAPI CRUD API for trainee management.

Assignment information: Assignment ID, Title, Description, Instructions, Start Date, Deadline, Assigned Batch, Created By, Status.

12 Module 8 — Submission

Trainee submits completed work. A submission can contain: description, GitHub repository, branch name, pull request, file/document, comments, submission date.

```
Assignment → Trainee Work → Submission → Mentor Review
→ Approved / Changes Required
```

13 Module 9 — Evaluation

Mentors evaluate trainee performance.

Category	Score
Technical Skills	/10
Problem Solving	/10
Coding Quality	/10
Git/GitHub	/10
Communication	/10
Teamwork	/10
Professionalism	/10

Evaluation result: Total Score, Percentage, Grade, Mentor Remarks.

14 Module 10 — Feedback

Mentors can provide feedback after: task, assignment, project, weekly review, monthly review.

Feedback fields: Strengths, Areas for Improvement, Recommendations, General Comments.

15 Module 11 — Dashboard

ADMIN • Total Trainees • Active Trainees • Total Mentors • Active Batches • Today's Attendance • Pending Tasks • Pending Reviews	MENTOR • My Trainees • Today's Attendance • Pending Tasks • Pending Submissions • Upcoming Deadlines • Recent Feedback	TRAINEE • My Profile / Batch / Mentor • Attendance % • Pending / completed tasks • Assignments • Performance • Feedback
---	--	---

16 Module 12 — Reports

Report	Columns
Trainee Report	Trainee, Batch, Mentor, Status, Joining Date
Attendance Report	Trainee, Total Days, Present, Absent, Leave, Percentage
Task Report	Task, Trainee, Deadline, Status
Performance Report	Trainee, Score, Grade, Feedback

17 Database Design

Initial tables: `users · roles · trainees · mentors · batches · attendance · tasks · assignments · submissions · evaluations · feedback`

```

USER
├── ADMIN
├── MENTOR
└── TRAINEE
BATCH → TRAINEES
MENTOR → TRAINEES
TRAINEE → ATTENDANCE, TASKS, ASSIGNMENTS, SUBMISSIONS, EVALUATIONS, FEEDBACK

```

18 Backend Architecture

```

backend/
├── app/
│   ├── main.py
│   ├── config/
│   ├── database/
│   ├── models/
│   ├── schemas/
│   ├── routers/
│   ├── services/
│   ├── auth/
│   └── exceptions/
├── tests/
├── requirements.txt
├── .env          ← never commit secrets
└── README.md

```

React → FastAPI Router → Validation → Service → SQLAlchemy → MySQL → Response

19 Frontend Architecture

```
frontend/  
├── src/  
│   ├── components/  
│   ├── pages/  
│   ├── layouts/  
│   ├── services/  
│   ├── hooks/  
│   ├── routes/  
│   ├── types/  
│   └── utils/  
├── public/  
└── package.json
```

20 Important APIs

```
POST /api/auth/login  
  
POST /api/trainees  
GET /api/trainees  
GET /api/trainees/{id}  
PUT /api/trainees/{id}  
DELETE /api/trainees/{id}  
  
POST /api/batches  
GET /api/batches  
GET /api/batches/{id}  
PUT /api/batches/{id}  
  
POST /api/attendance  
GET /api/attendance  
  
POST /api/tasks  
GET /api/tasks  
PUT /api/tasks/{id}  
  
POST /api/assignments  
GET /api/assignments  
POST /api/assignments/{id}/submit  
  
POST /api/evaluations  
GET /api/evaluations/{trainee_id}
```

DELETE /api/trainees/{id} deactivates the row. Do not immediately delete important records.

21 MVP — First Release

Do not build every feature immediately.

VERSION 1

Login → Dashboard → Trainees →
Batches → Mentors

VERSION 2

Attendance · Tasks · Assignments

VERSION 3

Evaluation · Feedback · Reports

22 Development Plan

Week	Focus	Work
1	Foundation	Python, FastAPI, Git, GitHub, MySQL, project setup
2	Backend	Database, SQLAlchemy, Trainee / Batch / Mentor CRUD, Postman
3	Frontend	React, Login, Dashboard, Trainee screens, API integration
4	MVP completion	Authentication, Roles, Validation, Testing, Bug fixing, QA review

23 Month-End QA

At the end of one month, QA checks:

FUNCTIONAL

Login, trainee CRUD, batch CRUD, mentor management, attendance, tasks, assignments.

API

Status codes, validation, authentication, authorization, error handling.

UI

Forms, navigation, responsive design, error messages, loading states.

DATABASE

Data creation, data updates, relationships, duplicate records, constraints.

24 GitHub Workflow

Requirement → GitHub Issue → Feature Branch → Development → Testing
→ Commit → Push → Pull Request → Mentor Review → Fix → Approval → Merge

Example branch: `feature/trainee-crud`. Example commit: `feat: add trainee CRUD APIs`.

25 QA Workflow

```

Developer → Developer Testing → Pull Request → Mentor Review → QA Testing
→ Bug Found?
  / \
Yes  No
  ↓   ↓
Fix  Approve
  ↓   ↓
Retest Release

```

26 Bug Classification

Severity	Meaning
Critical	System cannot operate.
High	Major feature does not work.
Medium	Feature works incorrectly but a workaround exists.
Low	Minor UI or usability issue.

27 Docker

After local development is stable:

React Container → FastAPI Container → MySQL Container

Use `Dockerfile` and `docker-compose.yml`. Basic command: `docker compose up`.

28 CI/CD

GitHub → GitHub Actions → Lint → Tests → Build → Docker Image → Deployment

29 Security Requirements

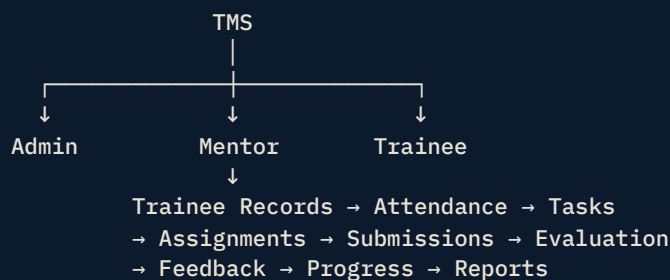
- Never commit passwords.
- Never commit .env.
- Never commit API keys.
- Never share private SSH keys.
- Hash passwords.
- Protect private APIs.
- Validate user input.
- Use role-based authorization.
- Use environment variables for secrets.

30 Definition of Done

A feature is DONE only when:

- Code is complete.
- Validation is implemented.
- API is tested.
- UI is tested where applicable.
- No critical errors remain.
- Git commit is created.
- Pull Request is submitted.
- Mentor review is completed.
- QA testing passes.
- Documentation is updated.

31 Final Project Flow



32 Final DTTP Outcome

Python → FastAPI → REST API → MySQL → React → GitHub → Testing → QA → Docker → CI/CD

More importantly, trainees should be able to work like a real development team:

Understand → Design → Develop → Test → Review → Fix → Deliver.

This TMS should be treated as the first production-style DTTP training project, with the scope expanded gradually rather than trying to build the entire system at once.